DEVOPS

LAB 04

Simple Python Database Connections

Name: Fatima Noor Ul Imran

Registration Number: 2023-BSCS-33

Course Code: BCS-649

Section: A

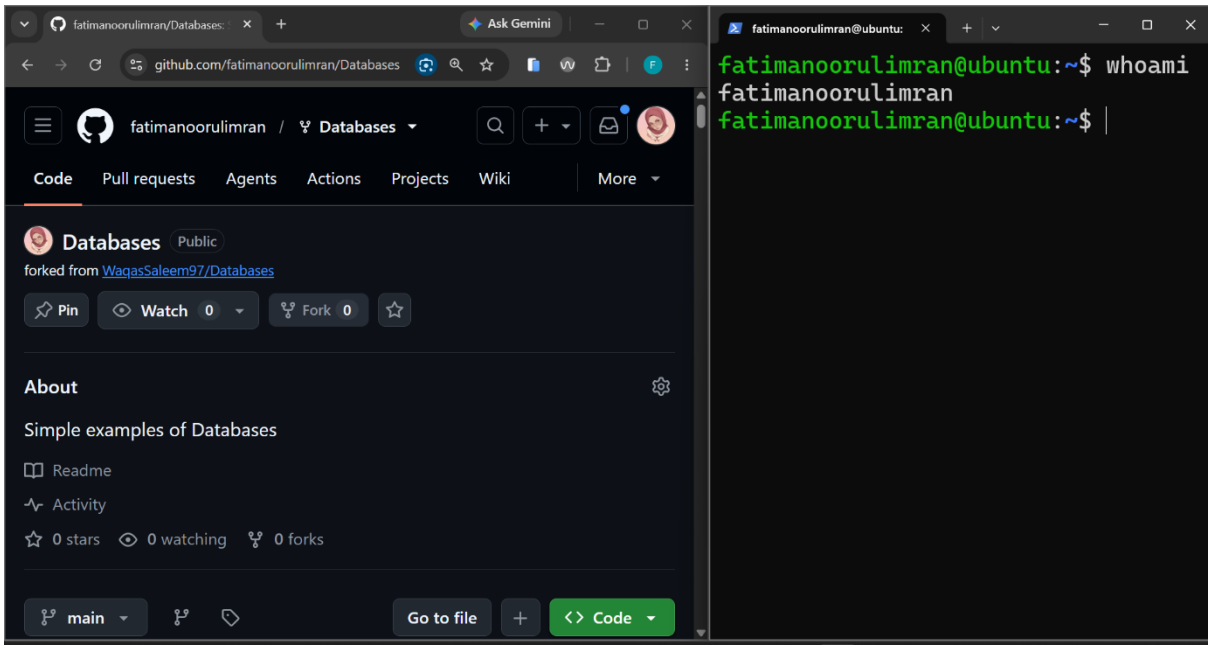
GitHub Username: fatimanoorulimran

Repository Links

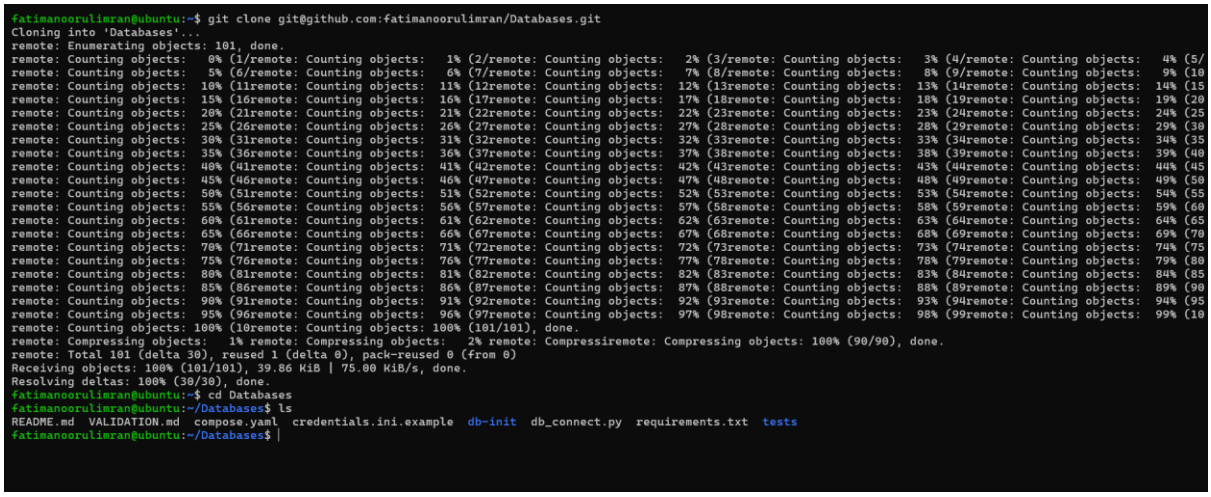
Databases fork: https://github.com/fatimanoorulimran/Databases

Submission directory: https://github.com/fatimanoorulimran/DevOps/tree/main/Labs/Lab04

Task 1: Fork and Clone the Databases Repository

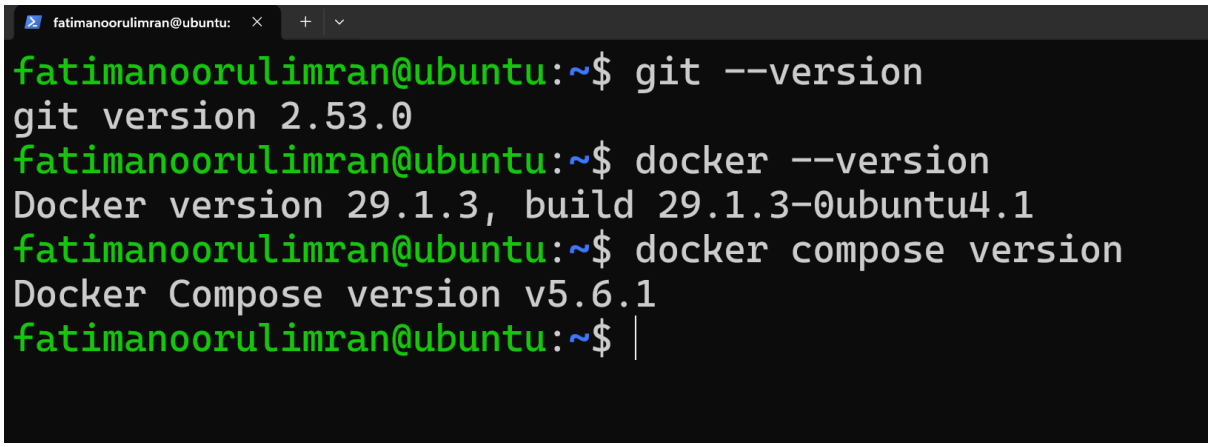


My fork fatimanoorulimran/Databases, forked from the instructor's Databases repository, next to my Ubuntu terminal.



My fork cloned over SSH into ~/Databases on the Ubuntu server.

Task 2: Install Docker, Docker Compose, and Python on Ubuntu

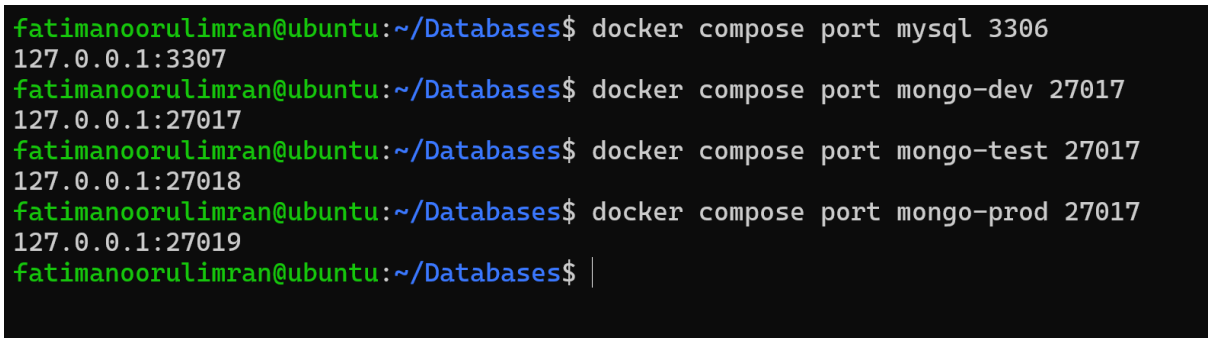


Git, Docker and Docker Compose installed and showing their versions after reconnecting.

Task 3: Start the Database Services



docker compose ps showing the mysql, mongo-dev, mongo-test and mongo-prod services healthy.



docker compose port output showing the host ports used by Python.

MySQL is reachable on host port 3307, MongoDB dev on 27017, test on 27018 and simulated prod on 27019.

My Ubuntu server runs Linux kernel 7.0, and the MongoDB 8.0 container refused to start with a message about an incompatible kernel (MongoDB issue SERVER-121912). I fixed it with a local compose.override.yaml that sets GLIBC_TUNABLES=glibc.pthread.rseq=1 for the three MongoDB services. The supplied compose.yaml was not changed, and the override file was not committed.

Task 4: MySQL with Values in the Code

```
(.venv) fatimanoorulimran@ubuntu:~/Databases$ python db_connect.py
Connected to MySQL: school_mysql
Server: 127.0.0.1 3307
Environment: dev
(.venv) fatimanoorulimran@ubuntu:~/Databases$
```

db_connect.py connecting to MySQL with the settings written in the code.

Task 5: MongoDB with Values in the Code

```
fatimanoorulimran@ubuntu:
(.venv) fatimanoorulimran@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) fatimanoorulimran@ubuntu:~/Databases$
```

db_connect.py connecting to MongoDB dev with the settings written in the code.

Task 6: MongoDB with os.getenv()

```
(.venv) fatimanoorulimran@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) fatimanoorulimran@ubuntu:~/Databases$
```

Environment-variable version connected to the dev database.

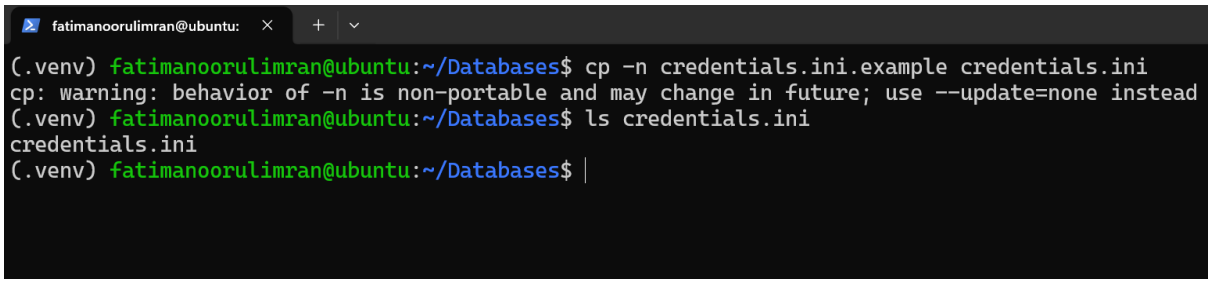
```
(.venv) fatimanoorulimran@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_test
Server: ('127.0.0.1', 27018)
Environment: test
(.venv) fatimanoorulimran@ubuntu:~/Databases$
```

The same unchanged program connected to the test database after exporting different values.

```
(.venv) fatimanoorulimran@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_prod
Server: ('127.0.0.1', 27019)
Environment: prod
(.venv) fatimanoorulimran@ubuntu:~/Databases$
```

The same unchanged program connected to the simulated prod database.

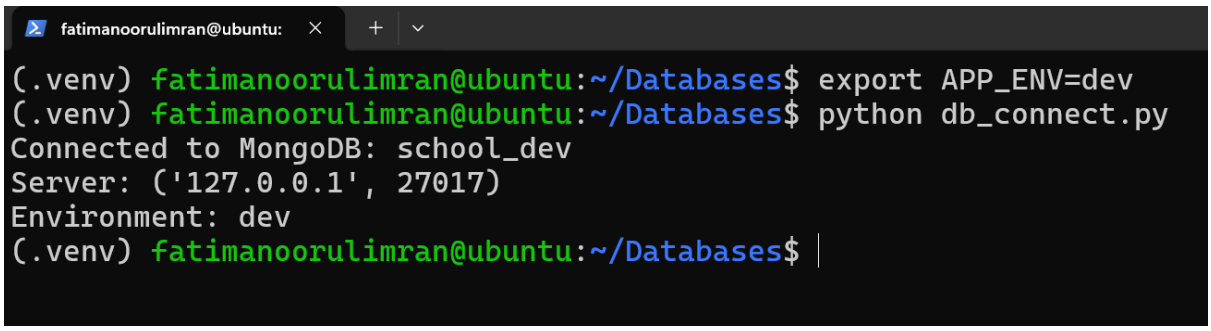
Task 7: MongoDB with a Credentials File



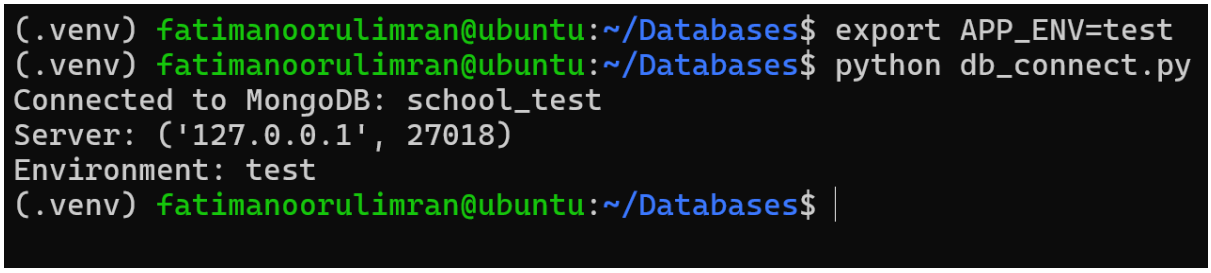
credentials.ini created from credentials.ini.example.



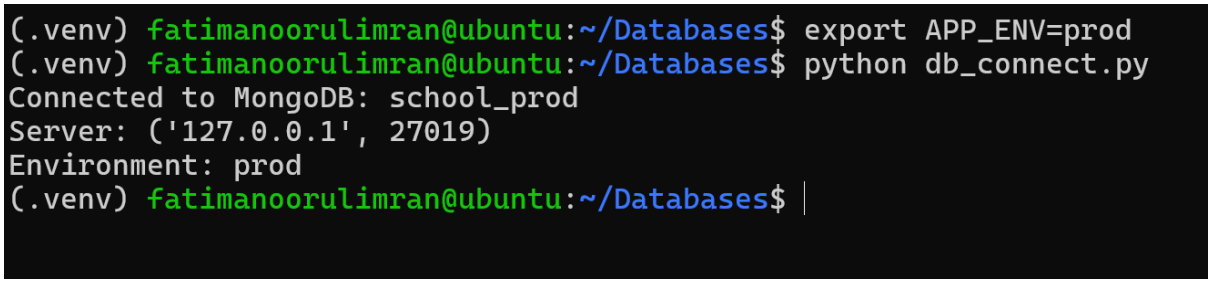
credentials.ini restricted to my account with mode 600.



Credentials-file version connected using the dev section.



Credentials-file version connected using the test section.



Credentials-file version connected using the prod section.

Final db_connect.py (credentials-file version)

```
import os
import configparser
from pymongo import MongoClient

config = configparser.ConfigParser(interpolation=None)
config.read("credentials.ini")
environment = os.getenv("APP_ENV", "dev")
username = config[environment]["username"]
password = config[environment]["password"]
endpoint = config[environment]["endpoint"]
database_name = config[environment]["database"]

uri = "mongodb://" + username + ":" + password + "@" + endpoint + "/" + database_name

client = MongoClient(uri, serverSelectionTimeoutMS=5000)
database = client[database_name]
database.command("ping")
print("Connected to MongoDB:", database.name)
print("Server:", client.address)
print("Environment:", database.environment_info.find_one()["name"])
client.close()
```

Exam Question 1: A Missing Environment Variable

```
(.venv) fatimanoorulimran@ubuntu:~/Databases$ python db_connect.py
Set all four MongoDB environment variables first.
(.venv) fatimanoorulimran@ubuntu:~/Databases$
```

With MONGO_DATABASE unset, the program stops with the missing-variable message.

Explain which os.getenv() call returned no value and why the if statement stopped the program.

os.getenv("MONGO_DATABASE") returned None because the variable no longer existed. The if statement checks that all four values are present, so not database_name was true and SystemExit stopped the program before the URI was built or any connection was tried.

Exam Question 2: Which Source Does the File Version Read?

```
(.venv) fatimanoorulimran@ubuntu:~/Databases$ unset APP_ENV
(.venv) fatimanoorulimran@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) fatimanoorulimran@ubuntu:~/Databases$
```

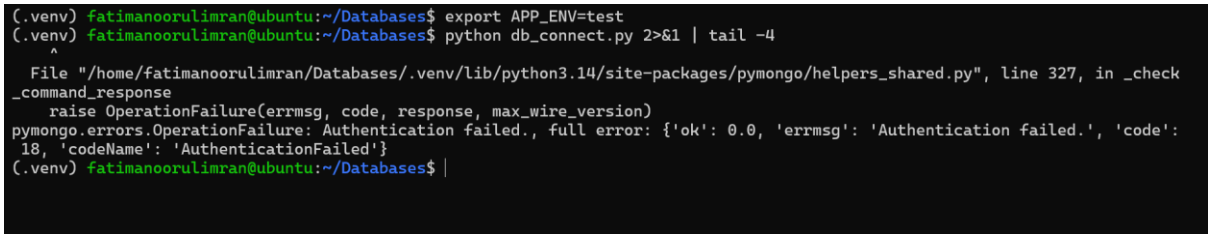
With APP_ENV unset, the credentials-file version connects to dev even though prod values are exported.

Explain why this version connects to dev even though the shell still contains the prod connection settings.

The file version never reads MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT or MONGO_DATABASE. It only reads APP_ENV to choose a section of

credentials.ini. With APP_ENV unset, os.getenv("APP_ENV", "dev") returns the default dev, so the dev section is used and the prod values in the shell are ignored.

Exam Question 3: A Wrong Password in the File



Authentication error after setting a wrong password in the [test] section of credentials.ini.

Explain why selecting the right endpoint does not make an incorrect password valid.

The endpoint and database name only say where to connect. MongoDB then checks the username and password against its own user records, so a correct endpoint cannot make an incorrect password valid.